

An FPGA-Based Hardware Emulator for Neuromorphic Chip With RRAM

Tao Luo^{ID}, *Member, IEEE*, Xuan Wang, Chuping Qu, Matthew Kay Fei Lee, Wai Teng Tang, Weng-Fai Wong, *Senior Member, IEEE*, and Rick Siow Mong Goh

Abstract—Neuromorphic chip with RRAM devices has been demonstrated as a promising computing platform for neural network-based applications. By directly mapping the weight matrices of neural networks onto RRAM-based crossbar arrays, high energy, and area efficiency can be achieved. However, the design of an RRAM-based neuromorphic chip faces many constraints due to the variability and limitations of RRAM. Simulation and emulation can help in the design of a neuromorphic chip prior to fabrication. However, software-based chip simulation on CPU is slow, especially for large-scale network-on-chip (NoC)-based chip design. In this paper, we present a hardware emulator on field-programmable gate array (FPGA) for an RRAM-based neuromorphic chip. Our emulator supports the emulation of static and dynamic variation of the RRAM-based crossbars used in the neural cores of a neuromorphic chip. Furthermore, an NoC is also implemented on FPGA to emulate the communication between the neural cores. Using the emulator, we show that effects, such as RRAM write and read noise and stuck-at faults affect the accuracy of an application on a neuromorphic chip. We also demonstrate the utility of the emulator in investigating NoC topologies, routing buffer depths, and neural core mappings.

Index Terms—Field-programmable gate array (FPGA) emulation, neuromorphic computing, RRAM, spiking neural network (SNN).

I. INTRODUCTION

DEEP neural networks prevail in a wide range of applications, such as machine learning and computer vision because of their human-level performance [1]–[3]. Due to their computation and memory-intensiveness, various kinds of computation platforms, such as GPUs and ASICs [4], [5], have been explored to accelerate neural networks. The implementation of a neural network can be divided into a training phase and an inference phase. The training phase is “one-off” and is usually performed on high throughput platforms such as

GPUs. The trained network is then deployed in the field to run in the inference phase.

With the increasing importance of neural network applications on mobile and embedded devices, finding efficient ways to deploy such networks has become a topic of intense research. For example, algorithmic approaches, such as compression and bit width reduction was explored to reduce energy consumption [6], [7]. Hardware and architectural design is another way to improve the efficiency of neural network inference. For example, the IBM TrueNorth neuromorphic chip was developed with an aim to reduce power consumption for the inference of spiking neural networks (SNNs) [8]. In this context, neuromorphic computing with RRAM-based crossbar has attracted a lot of research interest because of its remarkable energy efficiency as a dot product engine [9]. An RRAM-based crossbar takes a vector as its input and naturally outputs the result of a matrix-vector multiplication, which is the core computation in neural networks. Using the resistances of memristors to store the weight matrix, an RRAM-based crossbar shows great energy and area efficiency [10], [11]. Here, the weights are fixed and stored as “read-only” memory in the RRAM crossbar. Therefore, an RRAM-based neuromorphic chip is a promising computing platform for the inference of neural networks, especially for mobile or edge computing applications.

Despite the advantages of an RRAM-based neuromorphic chip over other computing platforms, there exist many challenges and constraints in the chip design process [12], [13]. First, the RRAM device suffers from various defects and variations, which may result in significant errors in neuromorphic computation [14]–[17]. These defects and variations mainly fall into two categories: 1) static defects and parametric variations such as stuck-at faults (SAFs) caused by fabrication process variation and 2) dynamic switching variation caused by driving circuits [14], [15]. SAFs are considered major static defects of RRAM arrays [15]. Due to poor control of the fabrication process, the resistance of an RRAM device could be stuck at a high or low resistance level, and cannot be changed further by the write process. Dynamic switching variation is a cycle-to-cycle variation that occurs during runtime. Any tiny fluctuation in the writing and reading current/voltage can lead to a large variation in the resistance of the memristor [14], [16], [17]. Moreover, even if the same driving voltage or current is repeated exactly, the RRAM still suffers from severe variation due to the inherent physical stochasticity of the resistive forming process. Thus, the effects of dynamic switching

Manuscript received July 21, 2018; revised October 17, 2018; accepted November 24, 2018. Date of publication December 25, 2018; date of current version January 18, 2020. This work was supported by the Singapore Government’s Research, Innovation and Enterprise 2020 Plan (Advanced Manufacturing and Engineering domain) under Grant A1687b0033. This paper was recommended by Associate Editor P. Leong. (*Corresponding author: Tao Luo.*)

T. Luo, X. Wang, C. Qu, M. K. F. Lee, W. T. Tang, and R. S. M. Goh are with the Institute of High Performance Computing, Agency for Science, Technology and Research, Singapore (e-mail: luo_tao@ihpc.a-star.edu.sg).

W.-F. Wong is with the School of Computing, National University of Singapore, Singapore.

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TCAD.2018.2889670

variation need to be carefully considered and evaluated during the design process. In this paper, we consider both static SAF and dynamic switching variations.

Like many nonvolatile memory technologies, RRAM also has endurance and retention issues. However, since we only consider the RRAM-based neuromorphic chip for inference only where the weights are read-only, the endurance requirement is different from that of RRAM devices used in general memory systems. According to [18], at 85 °C, the endurance of HfOx-based RRAM devices can reach up to 100 million times for write operations, and the retention can reach more than ten years. These suffice for an inference-only RRAM neuromorphic chip.

Second, because of fan-in and fan-out limitations of the RRAM crossbar, a neuromorphic chip has many neural cores each containing a crossbar with its peripheral circuitry. Communication between the neural cores is realized by a network-on-chip (NoC). NoC configurations such as the topology and the router buffer depth affect the performance of the neuromorphic chip and therefore, careful evaluation needs to be carried out before actual fabrication.

Typically, simulation and emulation are adopted to facilitate the chip design process before costly and nonrecurring chip fabrication. However, software-based simulation on CPU is slow, and can be two to three orders of magnitude slower compared with field-programmable gate array (FPGA) emulation if we take into account the variation of RRAM devices and NoC communication. Furthermore, a neuromorphic chip usually contains many neural cores each containing hundreds of neurons, resulting in a large NoC, thereby making software-based simulation even slower. Worse, it fails to account for physical realities, such as area, wire lengths, clocking, and other hardware characteristics. Hence, in this paper, we present an FPGA-based hardware emulator to accelerate the emulation as well as better match the reality of an RRAM-based neuromorphic system.

Our hardware emulator implements $N \times N$ crossbar-style neural cores on FPGA. We pipeline the operations in the crossbar for higher throughput and better hardware utilization. In order to evaluate the impact of dynamic switching variation of RRAM on the system-level performance, we also developed and implemented RRAM noise models on FPGA. In addition, static SAF is evaluated and discussed. Finally, an NoC was implemented for communication between the neural cores. With different configurations, we evaluate the impact of NoC on the overall performance.

Using MNIST as the benchmark, we design an SNN with an integrate and fire (IAF) neuron model. We show that both write and read noise affect the accuracy of the neuromorphic chip. The impact of write noise was especially significant. Without any mitigation, write noise can cause the accuracy to degrade to unacceptable levels. At the same time, we verify that the “write-read-check-write” method is an effective mitigation method to alleviate the impact of write noise. A tradeoff can be made between the number of rewrites and the resulting accuracy. For SAF, we find that *stuck-at-short* fault could cause severe accuracy degradation to the neuromorphic chip, which needs to be addressed before chip fabrication. We also

show that NoC configurations affect the overall performance of the neuromorphic chip significantly. A 2-D mesh topology is shown to be the best choice for neural cores with modest communication requirements. By trying different mappings, we find that throughput-aware mapping best suits the neural cores with NoC.

To the best of our knowledge, our hardware emulator is the first FPGA-based emulator for an RRAM-based neuromorphic chip that takes into account RRAM variations and also includes an NoC. Key contributions of this paper are: 1) we developed an FPGA-based hardware emulator for a complete RRAM-based neuromorphic chip. RRAM variation models are developed and implemented on FPGA to emulate the variations in the RRAM crossbars of the neuromorphic chip; 2) an NoC is implemented to emulate communication between neural cores in the neuromorphic chip; and 3) emulation considering both RRAM variation and NoC impact are conducted for an RRAM-based neuromorphic chip. Effects of the RRAM variation and NoC configuration are evaluated and discussed.

The rest of this paper is organized as follows. Section II introduces the basics of a neuromorphic chip, the variations in RRAM, and the NoC. Implementation of our hardware emulator is detailed in Section III. Experimental results are presented and discussed in Section IV. Lastly, Section V concludes this paper.

II. BACKGROUND AND RELATED WORK

A. Spiking Neural Networks

SNNs fall into the third generation of artificial neural network models [8], [19], [20]. In typical multilayer perceptron (MLP) networks, neurons generate outputs at each propagation cycle [21]. In contrast, neurons in the SNN fire only when a membrane potential reaches a specific value, which is an intrinsic quality of the neuron related to its membrane electrical charge. When a neuron in an SNN fires, it generates a signal which travels to other neurons which, in turn, increase or decrease their potentials in accordance with this signal. A first-order neuron model, such as the IAF model can be used in SNNs. A simple SNN with a single hidden layer is shown in Fig. 1(a). We can see that input and output of the SNN are both spike trains. The IAF neuron model is shown in Fig. 1(b). V_{th} in the figure represents the threshold voltage of a neuron. When the membrane voltage of the neuron exceeds this value, the neuron “fires” and a spike event is generated as its output. In essence, an SNN has a similar network structure as a traditional MLP but employs a different neuron model. Owing to the event-based computation of the neuron model in an SNN, it has potentially better energy efficiency as demonstrated by the digital IBM TrueNorth chip [8].

B. RRAM-Based Crossbar and Variation of RRAM

Owing to its low power consumption, nonvolatility, and small size, the memristor has been regarded as one of the most efficient ways to realize the synapse of the neural network. The weight of the synapse can be represented by the conductance of an RRAM memristor. RRAM devices are usually

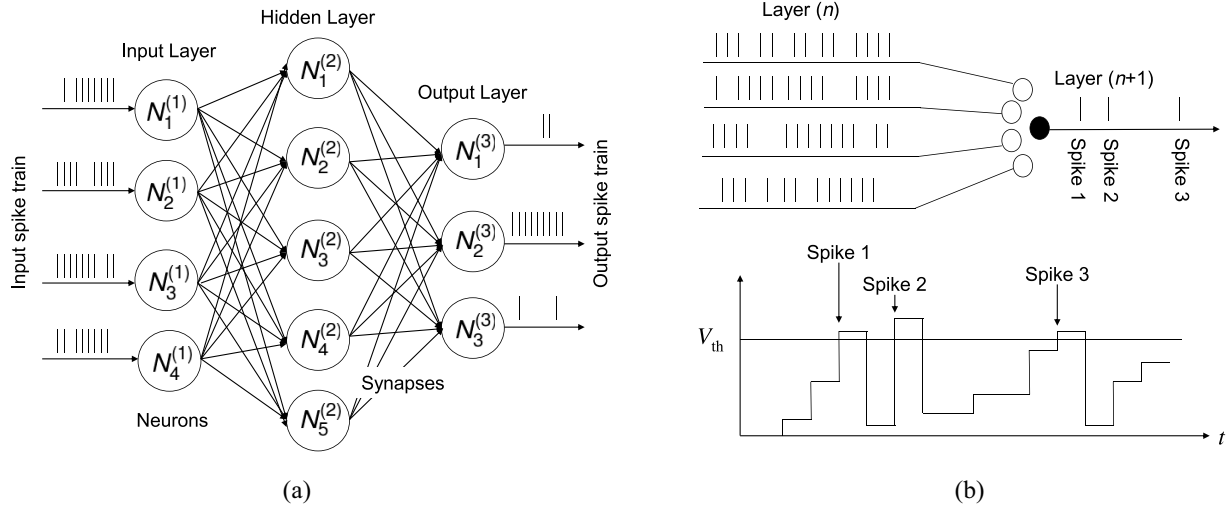


Fig. 1. Illustration of (a) SNNs with the (b) integrate-and-fire neuron model. The neuron membrane potential is increased in response to input spike events. When it exceeds the threshold value, the neuron fires and an output spike event is generated as shown in (b).

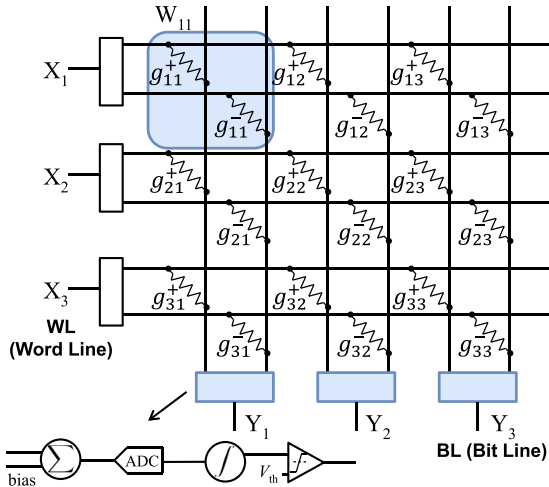


Fig. 2. Structure of an RRAM-based crossbar. In effect, two crossbar arrays are used in order to represent positive and negative weights. The current sums from two adjacent columns are digitized and accumulated to a buffer which is then compared to the threshold voltage in order to generate output spikes.

organized in a crossbar structure in a neuromorphic chip to enable neural network computation. Generally, an RRAM-based crossbar realizes the Dot-Product of a matrix-vector multiplication, $Y = W \cdot X + B$, where X , Y , and B are vectors and W is a matrix. Fig. 2 shows the structure of an RRAM-based crossbar example, where X , Y , and B are vectors of length 3 and W is a 3×3 matrix. As shown in Fig. 2, the word-line and bit-line are linked by a memristor in the cross point. When signals arrive, a voltage is applied to input X and current flows through memristors from word-lines to bit-lines. The conductance of the memristor will determine the amplitude of the current flow. Peripheral circuits of the crossbar including the analog-digital converters (ADCs) will digitize the current sum and add them to the bias B to obtain Y .

As we assume SNNs with an IAF neuron model in this paper, both the input and output of a neuron are binary

values, and they represent the existence or absence of spike signals. The RRAM-based crossbar in Fig. 2 realizes three post-synaptic spiking neurons, which share the three inputs from presynaptic neurons as the axons in the current crossbar array. The output of the neuron is generated by a current sum through the RRAM devices. Subsequently, each Y value of the neuron is digitized and accumulated to a buffer over time whose value is known as the *membrane voltage*. Once the *membrane voltage* of a neuron reaches a threshold voltage, the neuron will generate a spike signal as the output and then the value of the buffer is reset by subtracting the threshold voltage from it.

Note that since the weights in neural networks can be either positive or negative, two memristors are needed to represent the positive and negative values. The correspondence between the weight and conductance of the two memristors is

$$W_{ij} = \alpha\beta \cdot (g_{ij}^+ - g_{ij}^-) \quad (1)$$

where α is the gain of the ADC and β is the voltage applied to the row when there is a spike signal. Since the weight information is stored in the conductance of memristors, the effect of variation of RRAM is critical for the correct operation of neural networks.

In an RRAM-based crossbar, the RRAM device suffers from two kinds of variations, including static and switching variation [15], [22]. For static variation, SAFs is one of the most significant defects that affects RRAM operation severely [23]. As its name suggests, an RRAM device with SAFs has either a fixed low resistance or high resistance value due to fabrication defects, and the resistance of the device cannot be further changed even when a current is applied. If the RRAM device is stuck at a high resistance state, it is termed as *stuck-at-open*. Otherwise, if it is stuck at a low resistance state, it is called *stuck-at-short*.

For dynamic or switching variation, the effects are usually manifested as the set/reset and read variations of the memristor [16], [17]. In a set/reset process, the switching variability

of RRAM device causes deviation of the actual resistance from its expected resistance, which results in set/reset deviations [16]. In the read process, random telegraph noise (RTN) causes read variation, and is due to the structural, chemical, and electrical modification of the conductive filament (CF) or the localized current path [17]. The set/reset variation and read variation of a memristor can be regarded as write and read noise during weight access. The write and read noise may cause incorrect weights to be written or read, and may finally lead to an erroneous output in the RRAM-based crossbar. Thus, to emulate the switching variation of RRAM in an RRAM-based neuromorphic chip during weight access, we developed and implemented write and read noise models based on the set/reset and read variation of the memristor.

C. NoC

A neuromorphic chip comprises of neural cores connected by an NoC for intercore communication. Each neural core contains an RRAM-based crossbar and associated read/write circuits as well as circuitry for implementing the neuron model. Because a neural network can be mapped onto multiple neural cores in different ways to realize its computation, an NoC is adopted to enable scalable communication between the neural cores on the chip. Compared with traditional bus interconnects which are not scalable, communication can be improved by applying networking theory and methods to on-chip communication in an NoC.

One of the important properties of the NoC is its topology, which can be one of 2-D mesh, Torus, Ring, and so on. For different topologies, there are also different possible configurations, such as the buffer depth of the NoC routers. According to prior work on neuromorphic chips, communication between neural cores has an impact on the overall performance. Therefore, careful evaluation of NoC configurations needs to be carried out to optimize the overall performance. In terms of topology, neurogrid which is a mixed-analog-digital neuromorphic computing processor containing 16 neurocores, employs a tree topology using a custom router design with multicast capability [24]. In another system, the SpiNNaker platform achieves internode communication through a 2-D triangular toroidal mesh network [25]. Recent work by Carillo *et al.* [26] suggested using a hierarchical mesh-like NoC to increase throughput and to reduce communication latency. In this paper, we will focus on the mesh topology which is also used by the IBM TrueNorth chip [8]. The mesh network was pointed out to be suitable for neural network accelerators due to its scalability and ease of reconfigurability [27], [28].

The mapping of a network onto neural cores on the neuromorphic chip is also another important factor for overall performance. All the above-mentioned point to a need to have an emulator which can help to explore and evaluate design choices for NoCs.

D. Related Work

Due to the promise of high energy efficiency comparable to the brain, there has been a lot of work on

neuromorphic computing. For physiologically realistic emulation, Beuler *et al.* [29] and Moctezuma *et al.* [30] presented reconfigurable hardware to accelerate the emulation of neuromorphic computing, emphasizing on biophysical accuracy and biological compatibility with living organisms' behavior. For accelerating neuromorphic network simulation, Pande *et al.* [31] proposed a hardware modular neural tile architecture. The SNN memory requirement is reduced via a combination of fixed and configurable synaptic connections. In another work, Thomas and Luk [32] proposed an FPGA-based accelerator that increases simulation speed by simplifying the biological model.

Accelerators based on specialized hardware have also been proposed. Based on digital CMOS technology, the TrueNorth neuromorphic chip developed by IBM is one of the well-known ASIC accelerators [8]. Running on TrueNorth, multiobject detection and classification applications become highly energy efficient. With 400-by-240 pixel video input at 30 frames/s, the chip consumes only 63 mW. In order to further improve the energy and area efficiency, researchers are turning to emerging devices. Many RRAM crossbar-based neuromorphic designs and chips are being developed [9], [33]–[35]. For example, Hu *et al.* [9] proposed an RRAM-based crossbar for approximate matrix-vector multiplication, and achieved a speed-efficiency product of $1000\times$ to $10\,000\times$ compared to a custom digital ASIC. Park *et al.* [33] fabricated a 1 kb crosspoint synaptic RRAM array and CMOS-based neural chip, and validated the chip experimentally. Azghadi *et al.* [34] proposed a high performance nano-scale CMOS-memristive circuit, which mimics a number of essential learning properties of biological synapses. As a promising future memory technology, defects and variations of RRAM and methods to mitigate them have been studied extensively [14]–[17], [36]. However, to date there is no dedicated hardware emulator that targets the RRAM-based neuromorphic chip, and which considers both RRAM variation and NoC. To the best of our knowledge, this paper is the first FPGA-based hardware emulator that can evaluate both RRAM variation and NoC configuration of the RRAM-based neuromorphic chip.

III. FPGA-BASED HARDWARE EMULATOR

In this section, we describe in detail the design of our emulator, which contains three major parts, including crossbar-based neural cores, RRAM noise models, and the NoC.

A. Crossbar-Based Neural Cores

The architecture of our crossbar-based neural core implementation is shown in Fig. 3. The neural core has a size of 256×256 . Each of these cores contains 256 neurons, with each neuron having one output and sharing 256 inputs (axons). As shown in Fig. 3, the crossbar-based neural core consists of three modules, including a controller module, a Xilinx Block RAM (BRAM) module, and a crossbar module. The controller module is a finite state machine that controls and coordinates the operation of the BRAM and crossbar modules. The BRAM module, comprising BRAM resources on the FPGA, is used to store weights for the crossbar module. Computations of the

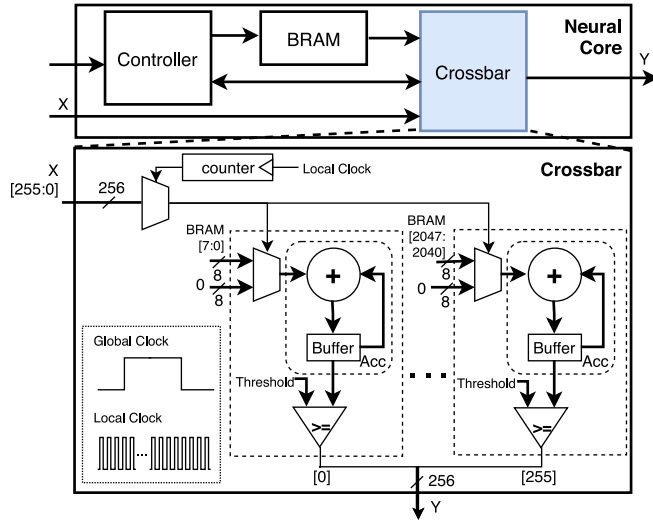


Fig. 3. Architecture of a neural core with 256 input and 256 output neurons.

crossbar are mainly realized by the crossbar module. In our design, each weight occupies a byte, resulting in a storage of 256 bytes for one 256-input neuron. For 256 neurons in a neural core, 64 KB are needed for weight storage without compression.

Implementation of the crossbar module is detailed in the blue box in Fig. 3. To balance performance and resource efficiency, in our design, we parallelize the computation of the 256 output or post-synaptic neurons in one neural core while serializing the computation for the 256-bit input. Since the bits in input X are independent of each other, this serialization is feasible, and correct input and output timing for the neural core is ensured. To coordinate the computation inside and outside the crossbar module, we introduce two clocks in our emulator, which are the local clock and the global clock, respectively. The local clock synchronizes the computation inside the neural core and the router in NoC, whereas the global clock synchronizes the NoC operation globally. Within one global cycle, all output spikes of each neural core are delivered to their destination cores. With this clocking scheme, the computation in each neural core and the spikes delivery in the NoC can be pipelined and synchronized. Note that input and output of the crossbar are both binary digits, so we only need an adder for each neuron, which results in 256 adders in total for a neural core. For the adder in each neuron, a buffer is equipped to store the temporary result. Combination of the adder and the buffer forms an accumulator, which is used to accumulate the result for each bit in input X . Each neuron is also equipped with a comparator that is used to compare the accumulated result with the given threshold value.

In every local cycle, 256 bytes are fetched from the BRAM module to the crossbar module, i.e., one byte for each neuron. Each bit out of the 256-bit input X is used to determine whether the accumulator adds the fetched weight to the buffer or not. After computations of all 256 bits in input X are done, comparators are used to obtain the final output for the neural core. Note that once a neuron generates a spike signal to the

Algorithm 1: Neural Core Processing

Input: $X[255:0]$, *Reset*
BRAM: $Weight[255:0, 255:0]$
Output: $Y[255:0]$
begin
 if *Reset* == True **then**
 $V_{buffer} = 0$;
 State = *StatePrepare*;
 else
 switch *State* **do**
 case *StatePrepare*
 $Index_{axon} = 0$;
 $SpikeIn[255:0] = X[255:0]$;
 go to *StateAccumulate*;
 case *StateAccumulate*
 for $Index_{neuron} \leftarrow 0$ **to** 255 **by** 1 **do**
 $V_{buffer}[Index_{neuron}] +=$
 $SpikeIn[Index_{axon}] \times$
 $Weight[Index_{axon}, Index_{neuron}]$;
 $Index_{axon}++$;
 if $Index_{axon} == 256$ **then**
 go to *StateCompare*;
 case *StateCompare*
 for $Index_{neuron} \leftarrow 0$ **to** 255 **by** 1 **do**
 if $V_{buffer}[Index_{neuron}] \geq Threshold$ **then**
 $SpikeOut[Index_{neuron}] = 1$;
 $V_{buffer}[Index_{neuron}] -= Threshold$;
 else
 $SpikeOut[Index_{neuron}] = 0$;
 $Y[255:0] = SpikeOut[255:0]$;
 go to *StatePrepare*;
 otherwise
 go to *StatePrepare*;
 End

output, the buffer will be reset. With such a design, the neural core can achieve a balance between high performance and high resource efficiency. Algorithm 1 shows the pseudo-code of the function implemented by a neural core.

B. RRAM Variation Models

In this section, we develop and implement RRAM variation models including an RRAM noise model and a static variation model. RRAM noise model accounts for the dynamic switching variation of RRAM. The static variation model emulates the static SAF defects of RRAM.

1) *RRAM Noise Model:* The RRAM noise model consists of write noise and read noise.

Write Noise: According to experiments on RRAM, resistance variation might occur when the memristor is being set or reset, and this introduces noise into the process of writing weights onto the crossbar architecture [16]. In this paper, we focus on set/reset variation of filamentary RRAM devices [37]. According to previous studies, this variation can be described by a lognormal distribution [38]. The resistance variation depends on the compliance current I_C , which is the maximum bounded current flow during the write procedure. The resistance range also depends on I_C , because it affects

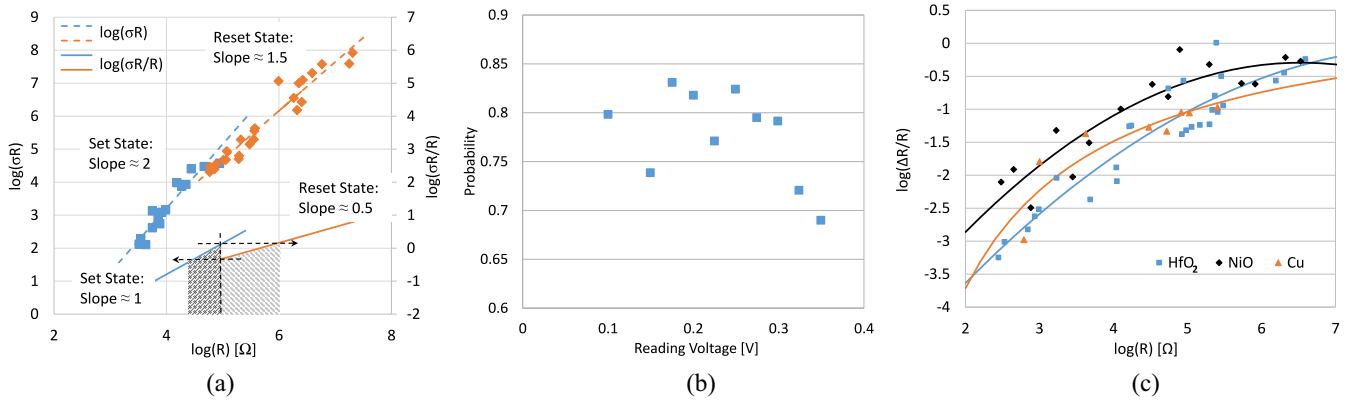


Fig. 4. Empirically measured write variation, read noise probability, and amplitude of read variation. Data from [16] and [17]. (a) σ_R for different R values due to write variation. (b) Read noise probability for different voltage. (c) Amplitude of RTN during read process.

the CF of the memristor. Therefore, there is a relationship between resistance variation and the resistance value. As an example, Fig. 4(a) shows the measured standard deviation σ_R for different resistances R for a typical HfO_2 -based RRAM (data from [16]). Fig. 4(a) also shows a linear behavior in the axes with a slope of ~ 2 for set states, and a lower slope of ~ 1.5 for reset states.

We apply a lognormal distribution to the written resistances of the memristors in our crossbar structure. This is transformed into weights in our emulator to emulate write noise. In a write procedure, in order to confine the write noise to a certain range, a write-read-check-write-verify method is usually adopted. In this method, the weight would be read back for verification after being written into the crossbar structure. If the noise is out of a targeted range, the weight would be rewritten until it falls within the targeted range. In our emulator, we also evaluate this method to analyze its effect on the final result.

Read Noise: Compared to write noise, read noise is harder to model because the weights are accessed numerous times in the computation. Read noise is introduced because of lattice defects along the CF surface of the memristor, thereby causing current flow to randomly fluctuate between two levels when a voltage is applied to a memristor. This manifests as a form of read noise known as RTN [17]. Therefore, in short, read noise occurs when the weights in the crossbar are read to perform a computation. Fig. 4(b) shows the empirically measured probability of read noise for different read voltages on our targeted memristor [17]. There is a relationship between the amplitude of variation and the resistance value as shown in Fig. 4(c), because both of them are dependent on CF size. The figure shows $\Delta R/R$ as a function of R for RRAM based on different materials, including HfO_2 , NiO , and Cu [17]. As resistance R increases, $\Delta R/R$ also increases and eventually saturates at larger R values.

We implemented our RRAM read and write noise model on FPGA and integrated the noise to the weight access procedure. The schematic of our RRAM noise model is shown in Fig. 5. As shown, our RRAM noise model is labeled as RRAM-NM and is composed of two parts: 1) the write noise model and 2) the read noise model.

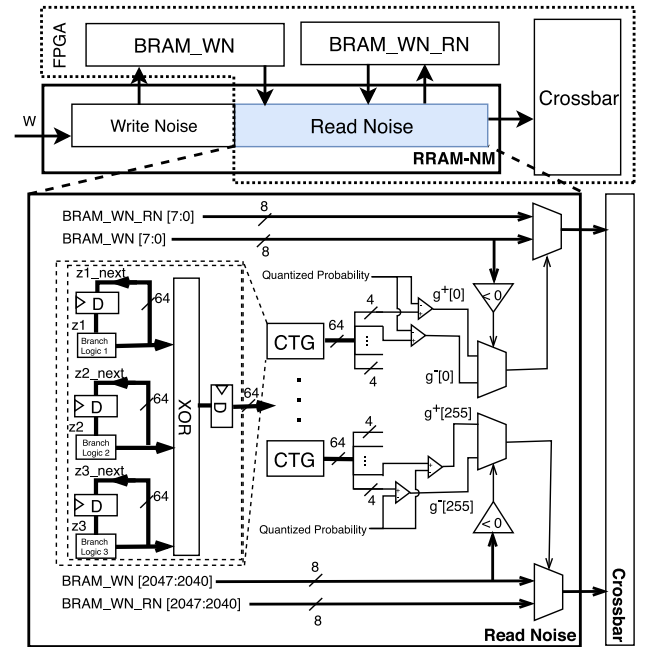


Fig. 5. Integration of RRAM noise with neural cores.

Whenever weights are to be written into BRAM, the write noise model is first applied to each weight according to the lognormal distribution as mentioned. Applying the read noise is more difficult because it is a cycle-to-cycle noise that needs to be considered for every read. We build the read noise in FPGA as shown in Fig. 5. Generally, for a single weight access, the read noise module generates two uniformly distributed numbers for the pair of memristors (g^+ and g^-) that are used to determine that weight. If the generated number is larger than the given read noise probability [see Fig. 4(b)], then read noise is ignored for this access, otherwise read noise will need to be generated for this read access. According to the amplitude data shown in Fig. 4(c), resistance variation of the memristors can be obtained, which can then be transformed into the read noise amplitude. As shown in Fig. 5, combined Tausworthe generators instead of traditional linear feedback shift registers

are used to generate uniformly distributed numbers with better randomness [39].

Every weight is represented with a pair of memristors whose resistances are fixed. As a result, the RTN amplitude, and hence the read noise amplitude is also fixed. In our case, for consistency and simplicity, we fixed the conductance of one memristor to its minimum value. For example, for positive weight, we set g^- to the minimum value supported by the RRAM device. For negative weight, g^+ is set to the minimum value instead. We store the weights with read noise on BRAM ahead of time and fetch them when necessary. Note that BRAM_WN in the figure indicates stored weights with write noise only, whereas BRAM_WN_RN indicates those with both write and read noise.

2) *Static Variation Model*: SAF is a static defect of RRAM. Due to its static property, it can be regarded as a special case of the RRAM write noise where the resistance value is fixed and cannot be changed by any write operations. For generality, we adopt a uniform random distribution for SAF in our RRAM array, i.e., each RRAM cell has the same probability of having an SAF defect. Following previous works [15], [23], for *stuck-at-short* defects, we varied the occurrence probability from 0% to 1%. For *stuck-at-open* defects, the occurrence probability was varied from 0% to 10%. The maximum resistance value and the minimum resistance value are assigned to RRAM cells with *stuck-at-open* and *stuck-at-short*, respectively.

C. On-Chip Communication

1) *NoC Model*: In this section, we explain in detail the NoC architecture and the various reconfigurability options available in our emulator. The basic unit of communication in NoC is a spike packet which encodes the following information: 1) destination core and 2) destination axon. The width of a spike is 13 bits with the first 5 bits representing the destination cores and the remaining 8 bits representing the destination axon. Each neuron core in the system is connected to a router and the routers are in turn connected to one other based on the NoC configuration. Fig. 6 shows the internals of the router. We model a simple two stage pipelined router with five ports. Port P0 is the port for the core interface, with the remaining ports P1–P4 for all the other four directions. When a router receives a spike from the core, the *encoder* is responsible for identifying the destination core and axon. It employs a look-up table with neuron ID as the key and the destination core and axon pair as the value. Following the encoding, the *decoder* is responsible for retrieving the target port from the routing look-up table which is populated at the beginning of the synthesis based on the NoC configuration and routing algorithm. There exists a *port controller* per port that controls the spike communication between the different ports. All the communication between the ports occurs in parallel due to duplication of resources. For example, port P0 can send a spike to port P1 while port P1 is sending a spike to port P2. In any given cycle, at most five possible spike communication can happen between the ports. The *arbiter* maintains a fixed priority among the ports with port 0 (the core) given the highest priority followed

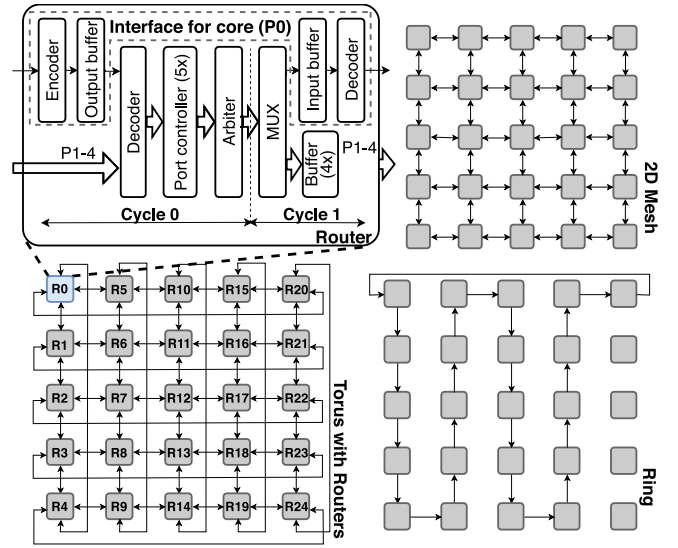


Fig. 6. Router architecture and examples of supported network topologies.

by the remaining ports. For example, when both port 0 and port 1 each contains a spike to be sent to port 2, by the virtue of our fixed priority arbitration, port 0 is granted access. We implemented peek flow control as described in [40] to control the traffic movement across the routers.

Our NoC model is highly reconfigurable to allow for evaluation of various design choices that impact the system performance. The following parameters are configurable.

Routing Topology: As discussed earlier, the router decodes the destination port by doing look-up on a pregenerated routing table. This flexibility enables the emulation of any custom routing. Currently, our framework supports 2-D mesh, 2-D Torus, and Ring topology (an example is shown in Fig. 6).

Buffer Depth: The buffer being modeled is a traditional *first-in first-out* buffer with a single entry per spike. There exists a buffer per port within the router. Each buffer has a read pointer and a write pointer to navigate through the entries. As will be discussed in Section IV, we see a significant performance improvement with deeper buffers.

Spike Width: The width of a spike packet is determined by the number of cores and axons in the system. We can configure the width based on the size of the chip being emulated. Currently, we emulate 25 cores and 256 axons per core, which requires 13 bits.

2) *Neuron Network-to-Neuron Core Mapping*: In addition to the NoC configuration, the mapping of a neural network onto the neural cores can have a significant impact on the performance of the system. This is because the performance is governed by the time required by the NoC to deliver all the spikes before the next iteration of computation commences. With our model, we provide a framework to evaluate the impact of different neural networks to neural core mappings.

IV. EXPERIMENTS

In this section, the impact of static and dynamic variations of RRAM and the different configurations of the NoC are evaluated by the emulator using the MNIST dataset

as a benchmark. With a fixed NoC configuration, different neural core mappings are evaluated and discussed. MNIST is a dataset of handwritten digits, containing 60 000 training examples and 10 000 test examples [41].

A. Experiment Setup

We implemented the emulator using Verilog HDL on Xilinx Virtex 7 XC7VX690T-2 FPGA (integrated onto an ADM-PCIE-7V3 board). Data communication between the board and host PC is realized using PCIe.

B. Soft Mapping SNN to Neural Cores

To evaluate the impact of RRAM variation and different NoC configurations on the performance of an RRAM-based neuromorphic chip with our emulator, we designed an MLP SNN with two hidden layers, which contain 512 and 256 spiking neurons, respectively. Fig. 7 shows the structure of our designed SNN. Each MNIST image has 28×28 gray pixels which, when flattened, is a 784 input array. However, the neural core model we adopt has a size of 256×256 , and is unable to take in all 784 inputs. Therefore, two additional layers of neurons are added in front of each hidden layer to split the inputs as shown in Fig. 7. Each hidden layer now consists of a split layer and a merge layer as shown in the right panel of the figure. For example, consider the first hidden layer. Due to the 256 fan-in constraint of each core, the 784 inputs are divided into four equal groups each with 196 inputs. As the original first hidden layer has 512 neurons, each of the 196-input group is thus connected to two cores (each with 196×256 connections) in the split layer to give a total of 512 neuron outputs. In total, the four input groups are connected to 4×2 neural cores. Subsequently, the output neurons of the 8 cores in the split layer are connected with a stride of 4 to the 8 cores in the merge layer in order for the receptive field of each neuron to cover the four input groups. Similarly, the 512 inputs to the second hidden layer are grouped into two groups so as to satisfy the 256 fan-in constraint. Two cores are each needed for the split and merge layers, respectively.

With such a *split-merge* structure, the SNN is mapped to 21 neural cores as shown in Fig. 7. The mapped neural network consists of five layers, each containing 8, 8, 2, 2, and 1 cores, respectively. To emulate the communication between the 21 neural cores, a 5×5 NoC with different configurations is also deployed in our emulator. Note that for input to our SNN, the pixel values of the benchmark images are coded as Poisson distributed spike trains. The output signals are spike trains in a one-hot format for the ten digit classes from 0 to 9. The class with the highest spike count is considered as the predicted answer. In the experiments, 35 time steps are used for the inference of each image.

C. Evaluation of RRAM Dynamic Variation

Table I shows the FPGA resources used by our emulator implementation. We can see that a neural core with noise integration doubles the usage of BRAMs and utilizes about 21% more LUTs and 76% more flip-flops than the one without noise integration. This is because extra logic and BRAMs are

TABLE I
REQUIRED FPGA RESOURCES FOR THE EMULATOR

	LUTs	Flip Flop (FF)	LUTRAM	BRAM
Single Neural Core Without Noise	12485	8750	0	28.5
Single Neural Core With Noise	15156	15383	0	57
2-D Mesh NoC Per Router	521	659	38	1
Torus NoC Per Router	588	686	48	1
Ring based NoC Per Router	450	644	40	1
2-D Mesh NoC with 21 Cores(Noise) (% of total resource)	339765 (78.4%)	352318 (40.7%)	1213 (0.7%)	1278 (86.9%)

TABLE II
EFFECTS OF RRAM DYNAMIC VARIATION AS WEIGHT ACCESS NOISE

	Average No. of Writes	Write Noise to Signal Ratio	Accuracy	WVR
No Noise	NA	NA	96.80%	0.00%
Read Noise Only	NA	NA	96.61%	30.49%
Write Noise Only	10	10%	95.83%	29.20%
	5	20%	89.40%	35.19%
	3	30%	62.31%	39.99%
	1	NA	1.33%	42.53%
Read & Write Noise	10	10%	95.77%	32.04%
	5	20%	90.17%	35.37%
	3	30%	66.40%	39.22%
	1	NA	1.32%	42.51%

required to process and store the noise information. Table I also shows the percentage of the on-chip FPGA resources utilized in total. In order to enhance the scalability of our emulator, we will investigate the implementation of NoC routers with physical copies using time multiplexing as our future work. Table II shows the impact of RRAM dynamic variation on the performance of the neuromorphic chip in terms of accuracy and a wrong vector ratio (WVR) metric. Apart from accuracy, the WVR metric is introduced to demonstrate the effect of noise. For this metric, the 10-bit output of the neural network at each time step is viewed as a vector. As SNNs use spike trains in their input and output, spike errors that occur at a few time steps may not necessarily affect the overall accuracy. Therefore, the WVR metric is used to reflect the difference in vectors between neural networks with and without noise. We take the output vectors from the neural network without noise as the baseline, and so its WVR metric is zero. write-noise-to-signal-ratio (WNSR) in the table indicates the range of write noise present, whereas “average number of writes” denotes the average number of write operations required to confine the noise to a specified range. We can make the following observations.

Write Noise: Write noise has a large impact on accuracy and the WVR metric. When WNSR increases, accuracy and WVR decrease rapidly and becomes unacceptable when the write-read-check-write method is excluded, i.e., when average number of writes is 1 as shown in Table II. The resistance range of memristors is from 100 k Ω to 1 M Ω . This indicates

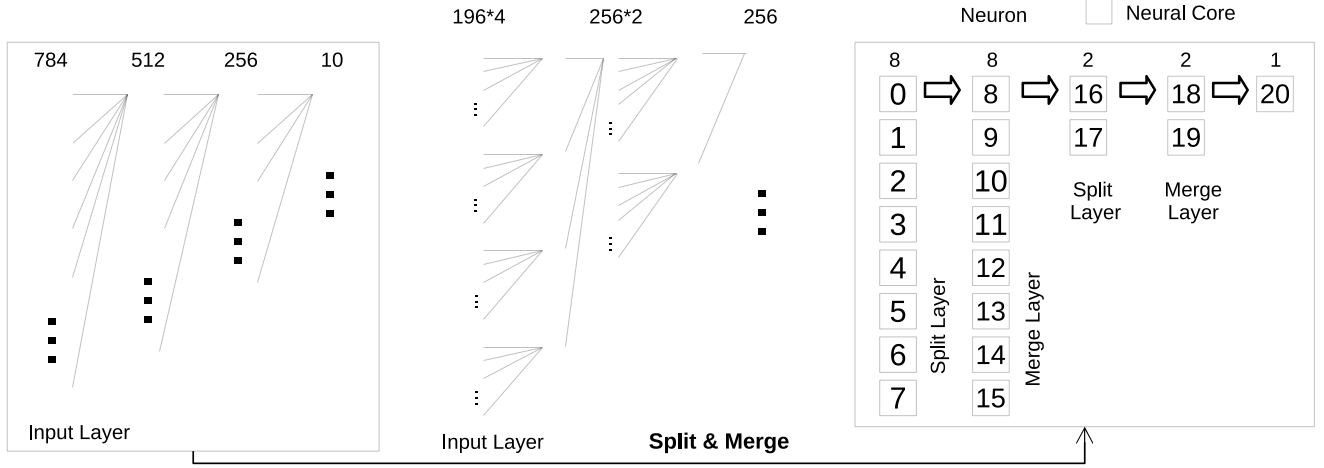


Fig. 7. Mapping of the MNIST SNN to neural cores with split-merge method to satisfy the fan-in constraint of each neural core. The network is mapped on to 21 neural cores in total.

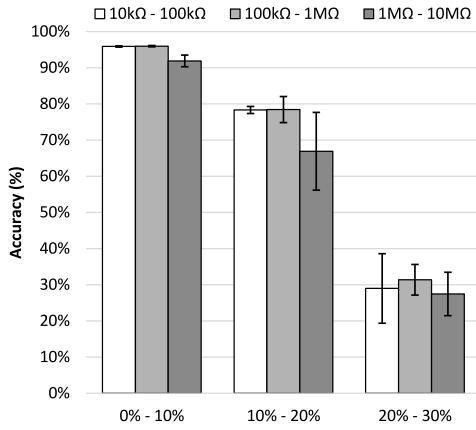


Fig. 8. Accuracy under different resistance ranges, and with different WNSRs.

that write-read-check-write method can be an effective mitigation method for write noise, where additional rewrites can help to alleviate its adverse impact on accuracy. Because rewriting leads to extra cost in terms of time and energy, for practical chip design, a tradeoff can be made between the number of rewrites and the resulting accuracy.

Fig. 8 shows the accuracy for different resistance ranges under different write-noise-to-signal ratios (WNSRs). With larger WNSR range, the accuracy decreases because of the increased write noise, which is in line with our expectation. From Fig. 8, it can be observed that the accuracy for resistance range 100 KΩ to 1 MΩ tends to be higher than other resistance ranges. This is because the write variation for this range ($\sigma R/R$) is lower as shown in Fig. 4(a).

Read Noise: Read noise also affects the accuracy and the WVR metric, but is not as severe as the write noise. As shown in Table II, read noise alone leads to about 0.19% degradation in accuracy, and ~30% in the WVR metric. This shows that an SNN is somewhat robust to spike errors. In this case, the resistance range of memristors is from 100 kΩ to 1 MΩ.

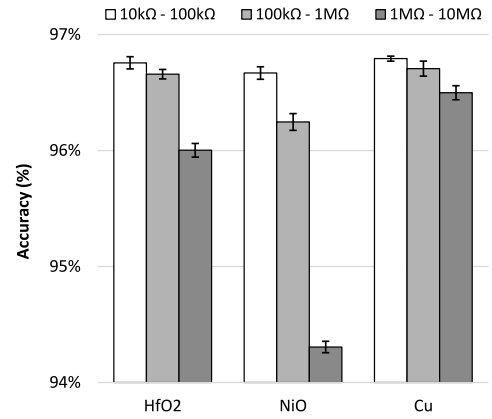


Fig. 9. Accuracy when read noise is present, under different resistance ranges and materials.

Both resistance range and RRAM material affect the read noise amplitude of the RRAM. According to Fig. 4(c), the larger the resistance value, the higher is the noise amplitude. Fig. 9 shows the accuracies for read noise under different resistance ranges and materials.

As shown, under a higher resistance range, the amplitude of read noise is increased which leads to the degradation of accuracy. Therefore, a lower operating resistance range is better for the sake of accuracy. However, a lower resistance range tends to incur higher energy consumption. Therefore, in practical chip design, a tradeoff between accuracy and energy consumption, as well as read noise tolerance should be carefully considered.

Read and Write Noise: When both write and read noise are present, the two noise either enhance the overall noise or cancel out each other to some degree. The results are also shown in Table II.

D. Evaluation of Stuck-at Fault in RRAM

As the most important static variation of RRAM, SAF defects has a considerable impact on a neuromorphic chip.

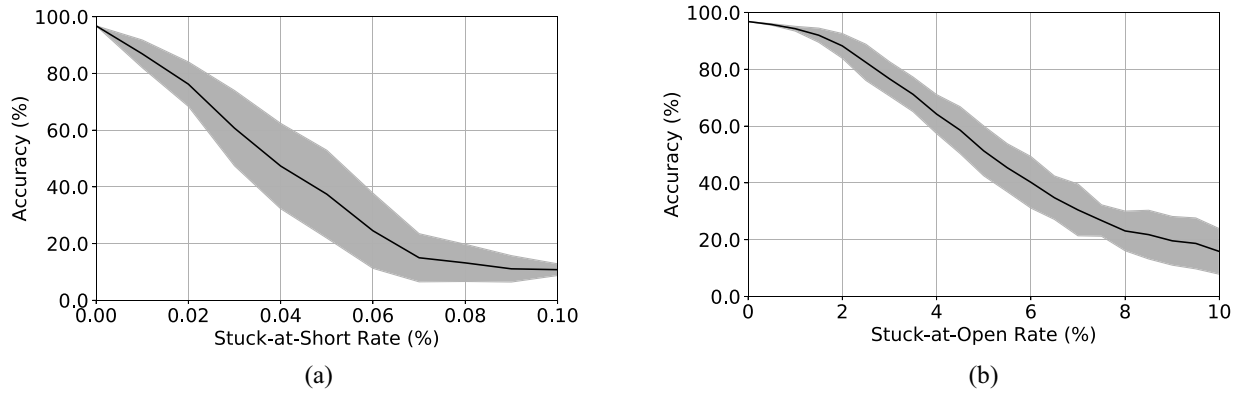


Fig. 10. Accuracy for stuck-at-short and stuck-at-open faults under different occurrence probabilities. (a) Accuracy for stuck-at-short fault under different occurrence probabilities. (b) Accuracy for stuck-at-open fault under different occurrence probabilities.

The impact of *stuck-at-short* and *stuck-at-open* on accuracy is shown in Fig. 10. We conducted ten sets of experiments on SAF defects for each data point. The solid line in the figure represents the average accuracy for different SAF occurrence probabilities, and the shaded regions in the figures represent the accuracy range for the ten sets of experiments. Compared with *stuck-at-open* defects, *stuck-at-short* has a much more adverse impact. A small number of *stuck-at-short* defects can degrade the accuracy dramatically. As shown in Fig. 10, less than 0.1% of *stuck-at-short* faults reduces the accuracy below 20%. This is because a *stuck-at-short* in an RRAM cell would lead to a *short circuit* in a column of crossbar, which would force the corresponding neuron to fire at every time step. Based on this observation, SAF affects not only the accuracy of the neuromorphic chip but also the power consumption. If the number of *stuck-at-short* is large enough, the power of the chip would become too large and could potentially cause thermal issues. Therefore, mitigation methods such as redundancy methods should be adopted to alleviate the impact of SAF.

E. Evaluation of NoC Configurations

To evaluate the impact of the NoC on the performance of a neuromorphic chip, we study two configurations that affect NoC performance: 1) the NoC topology and 2) the NoC router buffer depth. The required FPGA resources for implementing different NoC topologies are shown in Table I. In this evaluation, we use the minimum global clock period as a performance metric. The global clock period is measured in terms of the number of local clock cycles. In order to evaluate the theoretical performance of different NoC configurations, apart from the MNIST benchmark, we also evaluate the NoC by generating synthetic output spikes from each neuron with spiking rate of 100%, 50%, 25%, and 10%, respectively.

1) *NoC Topology*: The performance results of the NoC under three different topologies are shown in Fig. 11. The three most popular NoC topologies that we evaluated are the 2-D mesh, Torus, and Ring topologies. For all of these topologies, the 21 neural cores are sequentially mapped from the first to the fifth layers. As shown in Fig. 6, with the Ring topology, the 21 neural cores are connected end-to-end by a snake-like

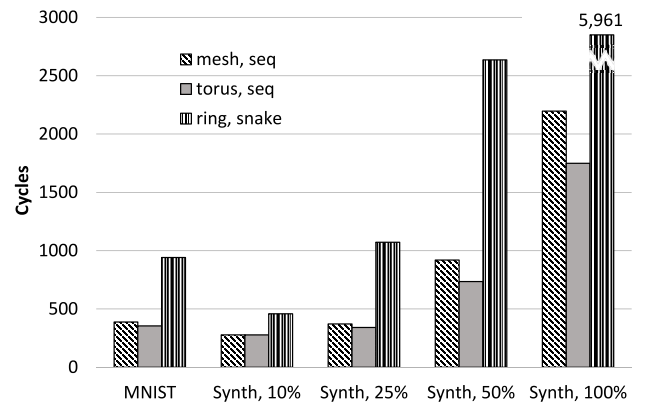


Fig. 11. Impact of the NoC topology on system performance (lower cycles has better performance).

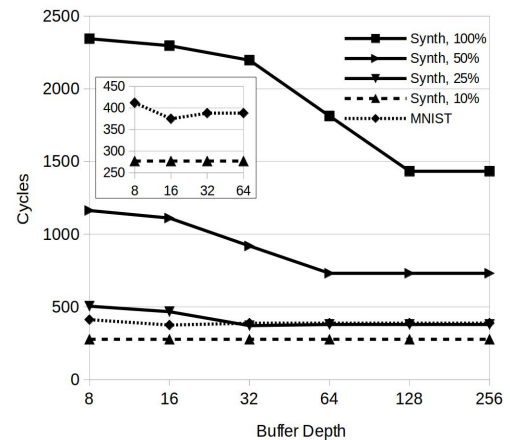


Fig. 12. Impact of NoC router buffer depth on system performance.

mapping, whereas for 2-D mesh and Torus, the 21 neural cores are mapped sequentially by following the router ID. Among the three topologies, for both MNIST and the synthetic benchmarks, Torus achieves the best performance in terms of having the lowest global clock period. As Ring has the worst performance compared to the other two topologies, we will focus our discussion on 2-D mesh and Torus. Torus has 9.6% higher performance than 2-D mesh for MNIST, and 0%,

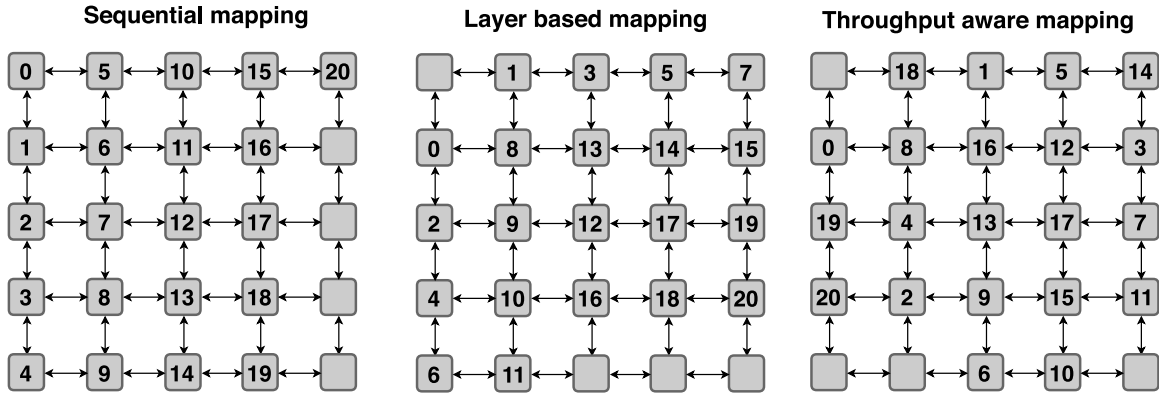


Fig. 13. Three different mappings of the software cores onto the neural cores on a 5×5 neuromorphic chip with NoC.

8.8%, 25.0%, and 25.6% better performance than 2-D mesh for each of the synthetic 10%, 25%, 50%, and 100% benchmarks, respectively. However, according to Table I, Torus incurs 13% more hardware resources than 2-D mesh, which corresponds to the fact that the Torus topology requires more links and a larger routing table than the 2-D mesh. In a practical neuromorphic chip design, such tradeoffs between area/energy overhead and system performance will have to be carefully considered.

2) *NoC Router Buffer Depth*: Fig. 12 shows the impact of NoC router buffer depth on system performance. Note that different buffer depths denoted on the x-axis refers to buffers for ports P1–P4 in the NoC router. For port P0 of the core interface, the buffer depth is fixed to 256. All the data in this section are collected assuming 2-D mesh topology and sequential mapping. Performance for the MNIST and synthetic benchmarks are shown for different buffer depths. Generally, with an increase in buffer depth, the system performance increases (i.e., lower global clock period in cycles), and eventually saturates after a certain buffer depth. For our SNN, a depth of 128 is sufficient for all the benchmarked situations since the performance of the Synthetic 100% benchmark saturates at this depth. For MNIST benchmark, the performance becomes saturated at a buffer depth of 32. The average spiking rate of the MNIST benchmark is deemed to be between 10% and 25% since the line is sandwiched between the Synthetic 10% and 25% benchmarks. For the MNIST benchmark, we observed a 3.5% higher performance for buffer depth of 16 than depths of 32 and beyond because of congestion which led to a longer critical path. For practical chip design, the buffer depth should be carefully determined to avoid unnecessary wastage of having a more conservative choice.

F. Neural Core Mapping

Efficient core mapping is critical for neural network performance on the neuromorphic chip. With our emulator, we explored three mappings for our SNN, including sequential mapping, layer-based mapping, and throughput-aware mapping.

1) *Sequential Mapping*: Sequential mapping is a baseline mapping where the software cores are mapped sequentially one by one onto the physical neural cores. There is no other knowledge considered during the mapping process. The

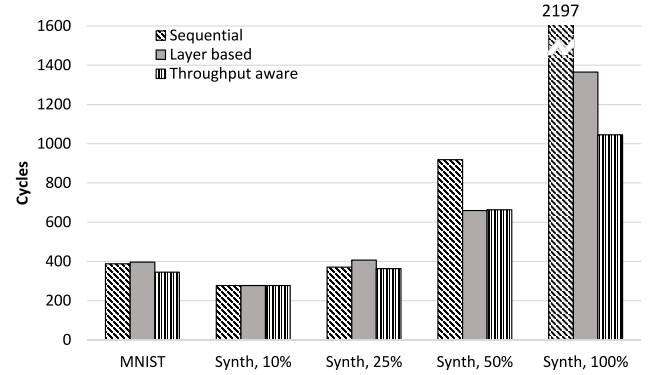


Fig. 14. Results of the (a) sequential, (b) layer-based, and (c) throughput-aware mappings. Lower cycles indicate better performance.

sequential mapping is shown in Fig. 13. The core numbers correspond to the software neural cores in Fig. 7.

2) *Layer-Based Mapping*: Layer-based mapping assumes knowledge of the layer information and how the neural cores are grouped into layers. During the mapping process, neural cores located within the same layer are mapped near to each other. A layer-based mapping is shown in Fig. 13.

3) *Throughput Aware Mapping*: The amount of input and output spikes of each neural core are considered in throughput-aware mapping. The neural cores that have large input and output pressure are mapped in the center of the NoC to alleviate the communication pressure. Fig. 13 shows the results of throughput-aware mapping.

Note that the three different mappings that are considered here are illustrative examples to demonstrate the utility of our emulator. The mapping of neural cores onto an NoC is a separate research topic, and there are other published works on this [42]–[44]. For these three mappings, their performance are shown in Fig. 14. Emulation results on our emulator show that throughput-aware mapping has the best performance among the three mappings.

G. Acceleration of FPGA Emulator Versus CPU Simulator

For the purpose of comparison with CPU simulation, we developed a C++ based simulator, which can perform the

same functionalities as our hardware emulator. Without simulating the effects of RRAM variations, emulation of a single neural core on the FPGA is $47\times$ faster than single-threaded execution on the CPU. When RRAM variations are included in the simulation, FPGA execution for a single neural core becomes $1458\times$ faster than single-threaded simulation on CPU. This remarkable improvement is because random number generation costs too much time on CPU. For the MNIST benchmark, we found that our FPGA emulator achieved $\sim 2000\times$ speedup compared with multithreaded execution on the CPU simulator,¹ both inclusive of RRAM variation simulation. The extra speedup compared with single core simulation is due to the software and memory overheads on the CPU.

V. CONCLUSION

In this paper, we proposed an FPGA-based hardware emulator to facilitate the design of an RRAM-based neuromorphic chip. We implemented neural cores, RRAM noise models, and an NoC on FPGA to emulate the operation, weight access, and communication between neural cores on a chip. Using MNIST as benchmark, we designed an SNN to evaluate the impact of RRAM variability and limitation in terms of accuracy and a WVR metric. We showed that the variability of RRAM is manifested as weight access noise, and significantly affects accuracy. In particular, without a mitigation method for write noise, the accuracy degrades to unacceptable levels. The impact of different NoC configurations on the communication between neural cores was also evaluated. In allowing simulation of RRAM variations and including NoC emulation, our emulator enables us to simulate entire applications faster, and with higher fidelity to make design choices, before building the actual neuromorphic chip.

REFERENCES

- [1] K. Fukushima and S. Miyake, "Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition," in *Competition and Cooperation in Neural Nets*. Heidelberg, Germany: Springer, 1982, pp. 267–285. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-46466-9_18
- [2] Y. LeCun *et al.*, "Handwritten digit recognition with a back-propagation network," in *Proc. Adv. Neural Inf. Process. Syst.*, 1990, pp. 396–404.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Las Vegas, NV, USA, 2016, pp. 770–778.
- [4] S. Chetlur *et al.*, "cuDNN: Efficient primitives for deep learning," *arXiv preprint arXiv:1410.0759*, 2014. [Online]. Available: <https://arxiv.org/abs/1410.0759>
- [5] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, 2017, pp. 1–12.
- [6] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," in *Proc. 28th Int. Conf. Neural Inf. Process. Syst.*, vol. 1. Montreal, QC, Canada, 2015, pp. 1135–1143.
- [7] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Quantized neural networks: Training neural networks with low precision weights and activations," *J. Mach. Learn. Res.*, vol. 18, no. 1, pp. 6869–6898, Jan. 2017.
- [8] P. A. Merolla *et al.*, "A million spiking-neuron integrated circuit with a scalable communication network and interface," *Science*, vol. 345, no. 6197, pp. 668–673, 2014.
- [9] M. Hu *et al.*, "Dot-product engine for neuromorphic computing: Programming 1T1M crossbar to accelerate matrix-vector multiplication," in *Proc. 53rd Annu. Design Autom. Conf.*, 2016, p. 19.
- [10] B. Govoreanu *et al.*, " $10\times 10\text{nm}^2$ Hf/HfO_x crossbar resistive RAM with excellent performance, reliability and low-energy operation," in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, 2011, pp. 1–4.
- [11] L. Xia *et al.*, "Switched by input: Power efficient structure for RRAM-based convolutional neural network," in *Proc. 53rd ACM/EDAC/IEEE Design Autom. Conf. (DAC)*, Austin, TX, USA, 2016, pp. 1–6.
- [12] J. Zhou, K.-H. Kim, and W. Lu, "Crossbar RRAM arrays: Selector device requirements during read operation," *IEEE Trans. Electron Devices*, vol. 61, no. 5, pp. 1369–1376, May 2014.
- [13] Y. Deng *et al.*, "RRAM crossbar array with cell selection device: A device and circuit interaction study," *IEEE Trans. Electron Devices*, vol. 60, no. 2, pp. 719–726, Feb. 2013.
- [14] S. Yu, Y. Wu, and H.-S. P. Wong, "Investigating the switching dynamics and multilevel capability of bipolar metal oxide resistive switching memory," *Appl. Phys. Lett.*, vol. 98, no. 10, 2011, Art. no. 103514.
- [15] C.-Y. Chen *et al.*, "RRAM defect modeling and failure analysis based on march test and a novel squeeze-search scheme," *IEEE Trans. Comput.*, vol. 64, no. 1, pp. 180–190, Jan. 2015.
- [16] S. Ambrogio *et al.*, "Statistical fluctuations in HfO_x resistive-switching memory: Part I—Set/reset variability," *IEEE Trans. Electron Devices*, vol. 61, no. 8, pp. 2912–2919, Aug. 2014.
- [17] S. Ambrogio *et al.*, "Statistical fluctuations in HfO_x resistive-switching memory: Part II—Random telegraph noise," *IEEE Trans. Electron Devices*, vol. 61, no. 8, pp. 2920–2927, Aug. 2014.
- [18] Y. Y. Chen *et al.*, "Endurance/retention trade-off on HfO₂/Metal cap 1T1R bipolar RRAM," *IEEE Trans. Electron Devices*, vol. 60, no. 3, pp. 1114–1121, Mar. 2013.
- [19] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Netw.*, vol. 10, no. 9, pp. 1659–1671, 1997.
- [20] S. Ghosh-Dastidar and H. Adeli, "Spiking neural networks," *Int. J. Neural Syst.*, vol. 19, no. 4, pp. 295–308, 2009.
- [21] S. Haykin and N. Network, "A comprehensive foundation," *Neural Netw.*, vol. 2, no. 2004, p. 41, 2004.
- [22] H.-S. P. Wong *et al.*, "Metal-oxide RRAM," *Proc. IEEE*, vol. 100, no. 6, pp. 1951–1970, Jun. 2012.
- [23] L. Xia *et al.*, "Stuck-at fault tolerance in RRAM computing systems," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 8, no. 1, pp. 102–115, Mar. 2018.
- [24] B. V. Benjamin *et al.*, "Neurogrid: A mixed-analog-digital multichip system for large-scale neural simulations," *Proc. IEEE*, vol. 102, no. 5, pp. 699–716, May 2014.
- [25] E. Painkras *et al.*, "SpiNNaker: A 1-w 18-core system-on-chip for massively-parallel neural network simulation," *IEEE J. Solid-State Circuits*, vol. 48, no. 8, pp. 1943–1953, Aug. 2013.
- [26] S. Carrillo *et al.*, "Scalable hierarchical network-on-chip architecture for spiking neural network hardware implementations," *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, no. 12, pp. 2451–2461, Dec. 2013.
- [27] T. Theodorides, G. Link, N. Vijaykrishnan, M. J. Invin, and V. Srikantam, "A generic reconfigurable neural network architecture as a network on chip," in *Proc. IEEE Int. SOC Conf.*, Santa Clara, CA, USA, Sep. 2004, pp. 191–194.
- [28] R. Emery, A. Yakovlev, and G. Chester, "Connection-centric network for spiking neural networks," in *Proc. 3rd ACM/IEEE Int. Symp. Netw. Chip*, San Diego, CA, USA, May 2009, pp. 144–152.
- [29] M. Beuler, A. Tchaptchet, W. Bonath, S. Postnova, and H. A. Braun, "Real-time simulations of synchronization in a conductance-based neuronal network with a digital FPGA hardware-core," in *Proc. Int. Conf. Artif. Neural Netw.*, 2012, pp. 97–104.
- [30] J. C. Moctezuma, J. P. McGeehan, and J. L. Nunez-Yanez, "Biologically compatible neural networks with reconfigurable hardware," *Microprocessors Microsyst.*, vol. 39, no. 8, pp. 693–703, 2015.
- [31] S. Pande *et al.*, "Modular neural tile architecture for compact embedded hardware spiking neural network," *Neural Process. Lett.*, vol. 38, no. 2, pp. 131–153, 2013.
- [32] D. Thomas and W. Luk, "FPGA accelerated simulation of biologically plausible spiking neural networks," in *Proc. 17th IEEE Symp. Field Program. Custom Comput. Mach. (FCCM)*, Napa, CA, USA, 2009, pp. 45–52.
- [33] S. Park *et al.*, "RRAM-based synapse for neuromorphic system with pattern recognition function," in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, 2012, pp. 1–4.
- [34] M. R. Azghadi, B. Linares-Barranco, D. Abbott, and P. H. Leong, "A hybrid CMOS-memristor neuromorphic synapse," *IEEE Trans. Biomed. Circuits Syst.*, vol. 11, no. 2, pp. 434–445, Apr. 2017.
- [35] M. Prezioso *et al.*, "Training and operation of an integrated neuromorphic network based on metal-oxide memristors," *Nature*, vol. 521, no. 7550, pp. 61–64, 2015.

¹Running on an Intel Xeon E5-2697v4 workstation.

- [36] S. Yu *et al.*, "A neuromorphic visual system using RRAM synaptic devices with sub-pJ energy and tolerance to variability: Experimental characterization and large-scale modeling," in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, 2012, pp. 1–4.
- [37] R. Degraeve *et al.*, "Causes and consequences of the stochastic aspect of filamentary RRAM," *Microelectron. Eng.*, vol. 147, pp. 171–175, Nov. 2015.
- [38] V. G. Karpov and D. Niraula, "Log-normal statistics in filamentary RRAM devices and related systems," *IEEE Electron Device Lett.*, vol. 38, no. 9, pp. 1240–1243, Sep. 2017.
- [39] P. L'Ecuyer, "Maximally equidistributed combined tausworthe generators," *Math. Comput. Trans. Amer. Math. Soc.*, vol. 65, no. 213, pp. 203–213, 1996.
- [40] M. K. Papamichael and J. C. Hoe, "CONNECT: Re-examining conventional wisdom for designing NoCs in the context of FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays*, 2012, pp. 37–46.
- [41] Y. LeCun, C. Cortes, and C. Burges, *MNIST Handwritten Digit Database*, vol. 2, AT&T Labs, Florham Park, NJ, USA, 2010. [Online]. Available: <http://yann.lecun.com/exdb/mnist>
- [42] F. Akopyan *et al.*, "TrueNorth: Design and tool flow of a 65 mw 1 million neuron programmable neurosynaptic chip," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 10, pp. 1537–1557, Oct. 2015.
- [43] Y. Ji, Y. Zhang, H. Liu, and W. Zheng, "Optimized mapping spiking neural networks onto network-on-chip," in *Proc. Int. Conf. Algorithms Archit. Parallel Process.*, 2016, pp. 38–52.
- [44] Y. Ji, Y. Zhang, W. Chen, and Y. Xie, "Bridge the gap between neural networks and neuromorphic hardware with a neural network compiler," in *Proc. 23rd Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2018, pp. 448–460.



Tao Luo (M'15) received the bachelor's degree from the Harbin Institute of Technology, Harbin, China, in 2010, the master's degree from the University of Electronic Science and Technology of China, Chengdu, China, in 2013, and the Ph.D. degree from the School of Computer Science and Engineering, Nanyang Technological University, Singapore, in 2017.

He is currently a Research Scientist with the Institute of High Performance Computing, Agency for Science, Technology and Research, Singapore.

His current research interests include high-performance computing with emerging device, artificial intelligence, reconfigurable computing system, and cyber security.



Xuan Wang received the bachelor's degree from the Electronics Department, Peking University, Beijing, China, in 2008 and the master's degree from the Electrical and Computer Engineering Department, National University of Singapore, Singapore, in 2012.

She was a Research Engineer with Keppel Offshore & Marine, Singapore, mainly responsible for sensor design, data-driven intelligent control, and remote monitoring on ships. She is currently a Research Engineer with the Institute

of High Performance Computing, Agency for Science, Technology and Research, Singapore. Her current research interests include hardware emulator/accelerator design, deep learning, and RRAM defects analysis.



field-programmable gate array.

Chuping Qu received the B.Eng. degree (Hons.) in electrical engineering from the National University of Singapore, Singapore, in 2017.

He did his university internship with Xilinx Asia Pacific Pte. Ltd., Bengaluru, India, for the characterization of memory devices in FPGA. He is currently a Research Engineer with the Institute of High Performance Computing, Agency for Science, Technology and Research, Singapore. His current research interests include neuromorphic computing and hardware implementation on



Matthew Kay Fei Lee received the B.S. degree in electrical and computer engineering from Carnegie Mellon University, Pittsburgh, PA, USA. He is currently pursuing the Ph.D. degree in computer science with Stanford University, Stanford, CA, USA.

He was a Research Engineer with the Institute of High Performance Computing, Agency for Science, Technology and Research, Singapore. He is interested in computer architectures enabled by emerging materials.



Wai Teng Tang received the B.Eng. (Hons.) and Ph.D. degrees from the National University of Singapore, Singapore.

He is currently a Research Scientist with the Institute of High Performance Computing, Agency for Science, Technology and Research, Singapore. His current research interests include high-performance computing, modeling and simulation, computer architecture, and artificial intelligence.



Weng-Fai Wong (SM'03) received the B.Sc. degree from the National University of Singapore, Singapore, in 1988 and the Dr.Eng.Sc. degree from the University of Tsukuba, Tsukuba, Japan, in 1993.

He is currently an Associate Professor with the Department of Computer Science, National University of Singapore. His current research interests include computer architecture, compilers, and high-performance computing.



Rick Siow Mong Goh received the Ph.D. degree in electrical and computer engineering from the National University of Singapore, Singapore.

He is the Director of the Computing Science Department, Institute of High Performance Computing, Agency for Science, Technology and Research, Singapore, where he leads a team of over 60 scientists in performing world-leading scientific research, developing technology to commercialization, and engaging and collaborating with industry.

His current research interests include artificial intelligence, high-performance computing, complex systems modeling and simulation, and human-machine interaction.